1. Modello Gerarchico (Web Browser): Il browser che utilizzi per navigare (come Chrome o Firefox) si basano su un sistema centralizzato chiamato PKI (Public Key Infrastructure).
- Autorità Centrali: La fiducia è stabilita da enti terzi chiamati Certification Authorities (CA).
- Fiducia "Dall'alto": Quando visiti un sito (es. della tua banca), il browser si fida del certificato del sito perché è stato firmato da una CA di cui il browser si fida già.
- Ruolo dell'utente: L'utente è passivo; non deve verificare personalmente l'identità del proprietario del sito, poiché il software lo fa automaticamente tramite le CA pre-installate.
2. Modello Decentralizzato: Web-of-Trust (GPG): GPG utilizza invece il Web-of-Trust (Rete di Fiducia), dove la sicurezza non dipende da un'autorità centrale ma dalle relazioni dirette tra le persone.
- Nessuna Autorità Centrale: Non c'è una "CA" suprema. La fiducia viene costruita "dal basso" attraverso le firme reciproche tra utenti.
- Verifica Personale: Per fidarsi di una chiave, GPG suggerisce di incontrare la persona dal vivo, verificare un documento d'identità e controllare il fingerprint (impronta digitale) della chiave su carta.
- Propagazione della Fiducia: Se io mi fido di una persona e firmo la sua chiave, e tu ti fidi di me, puoi scegliere di fidarti automaticamente anche delle chiavi che ho firmato io.

T4 - Tutorial: Usage examples of GPG

Goal of this tutorial

The goal of this tutorial is to demonstrate a practical usage of the public-key encryption and to introduce the web-of-trust for establishing the authenticity of public-keys.

GPG introduction

Many software tools are based on public-key encryption. Among them, we will see GnuPG (GPG). GPG is a complete implementation of the OpenPGP standard for sending and receiving secure emails. GPG implements both symmetric and asymmetric encryption ciphers and it is typically used for encrypting emails but it can also be used for encrypting generic input files (e.g., backups). There are many GUIs for facilitating the usage of GPG but in this specific case, we are interested in the baseline operations and procedures and therefore we use the command line.

Let's assume that there is a new user who is willing to set up a working GPG installation to communicate with friends. The first step is about generating a new key pair (i.e., public and private keys) for their new identity.

```
> gpg --gen-key
```

The default parameters used by GPG are fair but they can be tuned in case enhanced security is required (e.g., longer key sizes). Depending on the OS used for running GPG (and the specific version of the OS kernel), this operation could require a significant amount of time. This is because, in some implementations, the kernel needs to collect "enough entropy" for providing secure random numbers that are required for the key generation. If you are interested in the details, more info is provided in the course slides.

After some time, the keys are generated and then tagged with a specific identifier. **Why does gpg ask the user to enter a passphrase? What is it used for?**

It is possible to list all the public keys that are stored in our system (inside a "hidden" directory in the user's home directory) by using the following command:

> gpg --list-public-keys

A similar operation can be done for listing the available private keys:

> gpg --list-secret-keys

Just after creating the new key-pair, it is important to immediately generate and secure the corresponding **revocation certificate**. Please, try to consider why a revocation certificate is fundamental. (In some recent versions of gpg the revocation certificate is automatically generated upon the key-pair generation). Please read carefully the warnings that are displayed when running the following command:

> gpg --output revoke.asc --gen-revoke <KEY-ID>

Now our brand new public-key is ready to be used but… no one knows it! That's a problem! **We need a way for delivering our public-key to all our existing (and future) contacts**. A first simple approach would be to publish our public-key in a "secure way". **What do we exactly mean with "secure way" in this specific case? What are the relevant security properties to be considered?**

We could make available the public-key in a webpage but the website must support the https protocol. Why? What's wrong with http? Minor problem: the generated public-key is a binary file. Likely a text file would be easier to be published on the web:

> gpg --export -a <KEY-ID> > mykey.asc

In the exported file, there is the text version of the public-key. The private-key remains stored (and well protected?) in my home directory. **Question: how is the access restricted to my home directory? Is it secure or not? Is the home directory content encrypted? How can I effectively manage the confidentiality of my private-key?**

Another approach for making my public-key available to my friends is to use a key server. That is an infrastructure that is specifically dedicated to the publication of public keys and their management.

> gpg --send-keys --keyserver <KEY-SERVER> <KEY-ID>

This operation uploads the newly created public-key to a specified key server. We will **NOT execute this operation** since the key-pair that we have created is not related to a real human being but it is only for testing purposes.

Obviously, it is also possible to query the key server for finding already existing public keys. For example, the query could be based on the email address that is linked to the public-key.

```
> gpg --search-keys 'g.dangelo@unibo.it'
```

If the output of the command reports the public-key that I'm looking for then I can import it into my keyring. **Warning: think carefully about the security implications of this action**. In the following, we will discuss this issue again.

After importing the public-key, I can verify that it is now included in my keyring.

```
> gpg --list-public-keys
```

Now, we use the newly imported key to encrypt a document that needs to be delivered in a confidential way (to the owner of the public-key):

```
> gpg --encrypt --recipient 'g.dangelo@unibo.it' foo.txt
```

Clearly, it is possible to also encrypt binary data and not only text files.

If it is not necessary to hide the content of the message but only to authenticate its originator (and to guarantee its integrity) then we can proceed in this way:

```
> gpg --output foo.txt.sig --sign foo.txt
```

As seen in the slides about public-key encryption, in this case, the sender does not need to use the public-key of the recipient (but the recipient will need the public-key of the sender to validate the authenticity of the received message).

It is also possible to combine both operations to get confidentiality, authenticity, and integrity:

```
> gpg --encrypt --sign --recipient 'g.dangelo@unibo.it' foo.txt
```

Web-of-Trust

Now we can consider again the problem related to the authenticity of the public-key that we used for encrypting our message to preserve confidentiality. The mechanism that is typically used by gpg is **different from the approach followed in web browsers** (that we will see in the

following of this course) and it is called "**web-of-trust**". The basic idea of the web-of-trust is that after carefully checking the identity of the people that we know (in real life) and their matching with their corresponding public-key, then we can validate this match by signing their public-key (using our private-key). In practice, signing a key of someone has the effect of extending our web-of-trust and moreover, maybe we can accept to trust someone that is unknown to us but that has been validated by one or more friends of ours. Please note that, in this case, the number of friends that validate the public-key of someone that I don't know affects my perception of trust.

Since it proves quite inconvenient to bring with us the full printed version of our public-key (i.e., it is quite long), it is preferred to use the fingerprint (i.e., a hash value) of the public-key. This is generated using the following command:

```
> gpg --fingerprint <KEY-ID>
```

The suggested procedure to follow is: print out on paper the fingerprint and the identifier of the public-key. All of this is then provided to the person who is willing to sign our public-key. This operation should happen only after the verification of an identification document.

When all these operations are completed then it is possible to get the key to be signed using:

```
> gpg --keyserver <keyserver> --recv-keys <KEY-ID>
```

Now, after a final check to verify that the information on the public-key is as expected, it is possible to proceed with the key signature:

```
> gpg --sign-key <KEY-ID>
```

At this point, the signing operation is stored only locally and it needs to be propagated to a key server for really building a public web-of-trust:

```
> gpg --keyserver <keyserver> --send-key <KEY-ID>
```

It is possible to list all the signatures that have been collected by a specific public-key using the following command:

```
> gpg --list-sigs <KEY-ID>
```

Note: if this command reports only self-signatures for a given key then this is due to a change of behavior in the signing of keys that has been implemented to mitigate the problem of fake keys submitted to key servers. The actual procedure being implemented to create a web-of-trust is

more complex than the one reported in this tutorial but the theoretical framework discussed here is still valid.

Using GPG for symmetric encryption

Even if GPG is typically used for public-key encryption, it also implements some symmetric ciphers. For example:

```
> gpg --output foo.txt.gpg --symmetric foo.txt
```

This operation encrypts an input file using the default symmetric cipher provided by GPG. Clearly, in this case, GPG asks for a secret key that needs to be shared with the recipient of the encrypted file.

The different symmetric ciphers provided by GPG can be listed with:

```
> gpg --version
```

AGE

The usability of GPG is quite limited and therefore many other tools have been proposed for encrypting messages and files. In many cases, they also have the goal to reduce the attack surface that in the case of GPG is too large. Among them, age is a very promising tool that is implemented using the Go programming language. It is cross-platform (Windows, GNU/Linux, Mac) and it is already part of many Linux distributions. For a list of usage examples please see the age web page.

References

- The GNU Privacy Handbook
- Web of trust (Wikipedia)